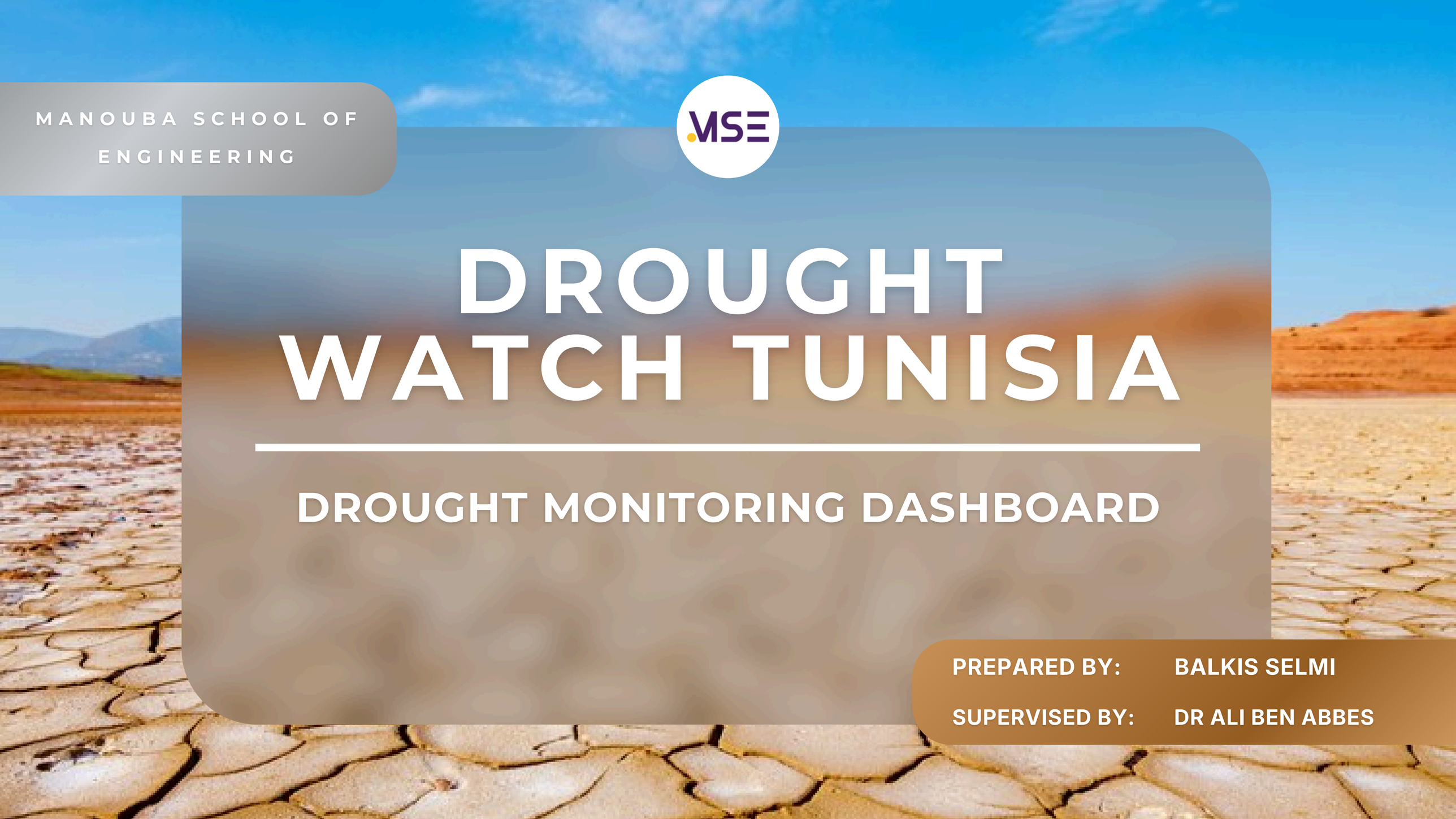


MANOUBA SCHOOL OF ENGINEERING

DROUGHT WATCH TUNISIA

DROUGHT MONITORING DASHBOARD

PREPARED BY: BALKIS SELMI

SUPERVISED BY: DR ALI BEN ABBES

Table of Contents

1 Introduction

2 Methodology

3 System Architecture

4 Dashboard Overview

5 Added Value

6 Limitations

7 Future Work

8 Conclusion

1.INTRODUCTION

General Context

- 8 drought years in last decade
- Dam storage ~28–29% capacity (2025)
- Rainfall ~110M m³ vs historical ~520M m³

Solution

interactive web-based dashboard:

- visualizes drought indicators
- analyzes drought behavior across space and time

Objective

Support drought monitoring and decision-making in Tunisia.





2.METHODOLOGY

The methodology of this project is based on a structured workflow.



Data Preparation



Backend API for data access



Frontend visualization (map + charts)



User interaction and analysis (click, statistics, export)

3.SYSTEM ARCHITECTURE



Frontend
Interactive interface (map, chart, control**s)**

Backend
API handling data requests and processing

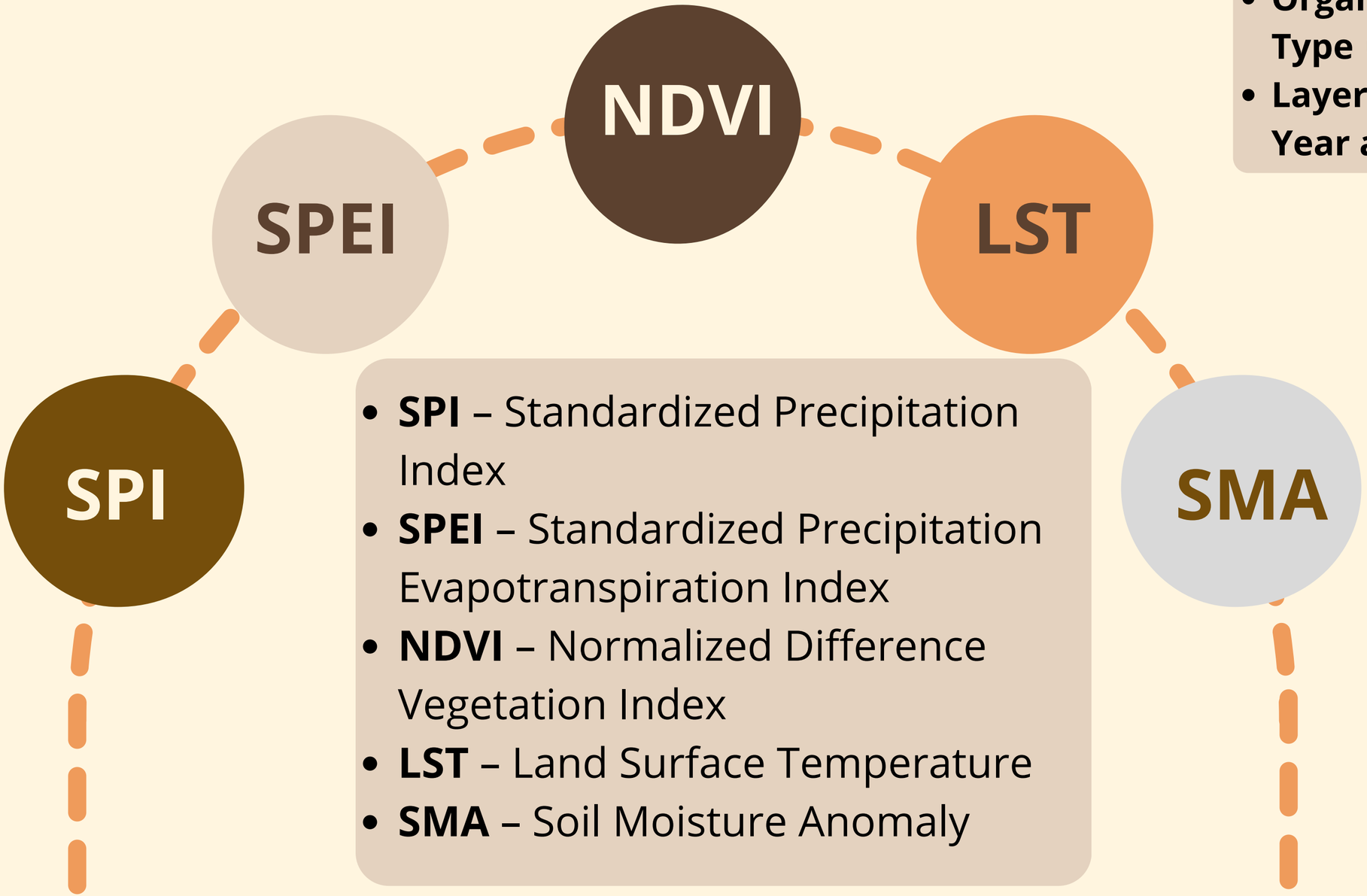


Data Layer
Organized drought indicator datasets



3.1 DATA LAYERS

- **Organized by Indicator Type**
- **Layers names contain Year and month**

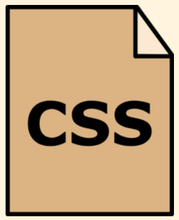


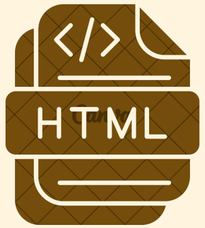
- **SPI** – Standardized Precipitation Index
- **SPEI** – Standardized Precipitation Evapotranspiration Index
- **NDVI** – Normalized Difference Vegetation Index
- **LST** – Land Surface Temperature
- **SMA** – Soil Moisture Anomaly

3.2 BACKEND

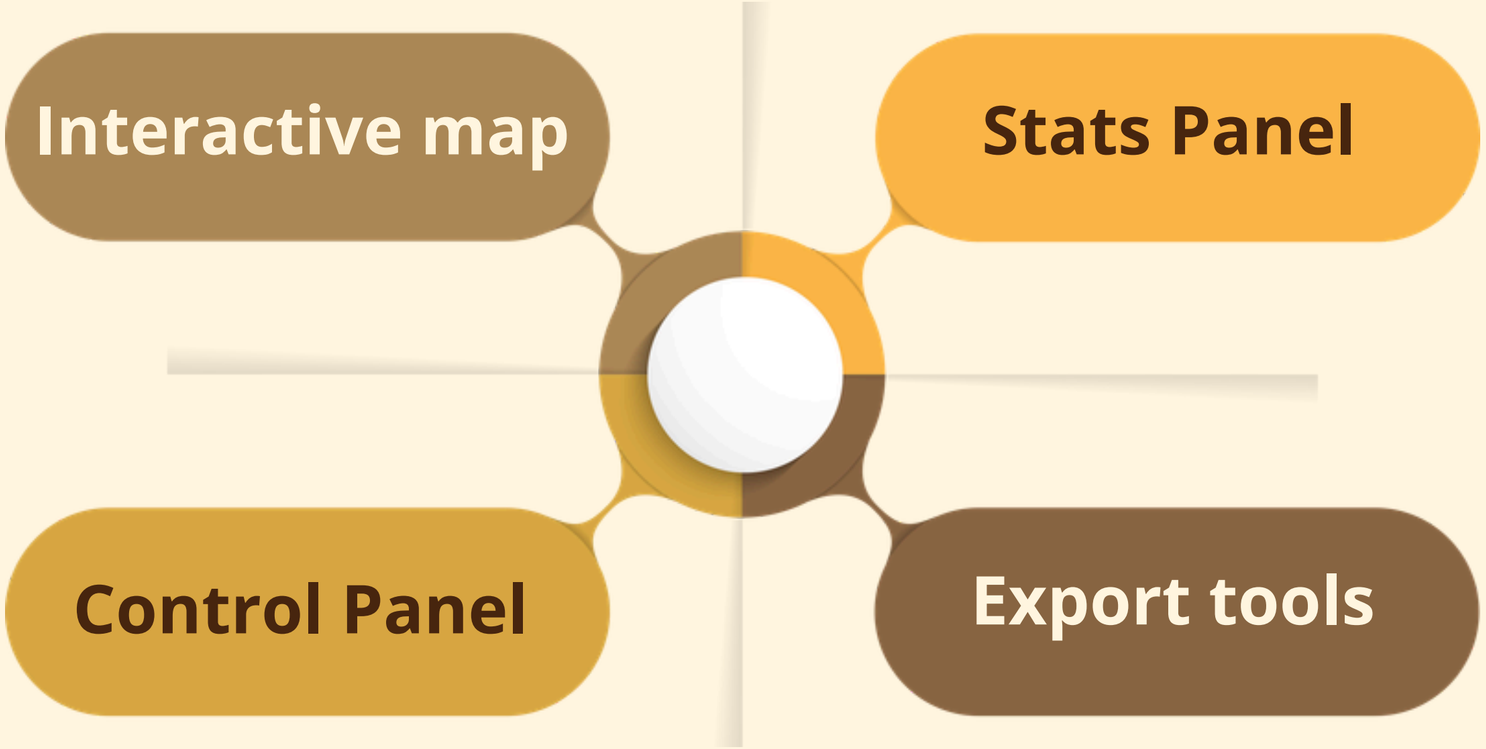
Functionality	Code Function(s)	Libraries Used
Server Initialization & Configuration	Flask(__name__), CORS(app), BASE_DIR, DATA_DIR, LAYER_REGISTRY	flask, flask_cors, pathlib
Automatic Layer Discovery	discover_layers_auto_only()	pathlib, re, rasterio, numpy
Layer-Specific Nodata Configuration	LAYER_NODATA_CONFIG	—
Nodata Values Retrieval	get_nodata_values_for_layer(layer_name, src)	rasterio, numpy
Data Validation	is_valid_value(val, nodata_values, layer_name)	numpy
Configuration API Endpoint	/api/config → get_config()	flask.jsonify
Spatial Extraction (Map View)	/api/data/<layer>/<year>/<month> → get_data()	rasterio, numpy, flask
Temporal Extraction (Time Series)	/api/timeseries/<layer>/<lat>/<lon> → get_timeseries()	rasterio, numpy, flask
Statistical Computation	np.min(), np.max(), np.mean(), np.polyfit() (within get_data() & get_timeseries())	numpy
JSON Response Formatting	jsonify(response_data)	flask, json

3.3 FRONTEND

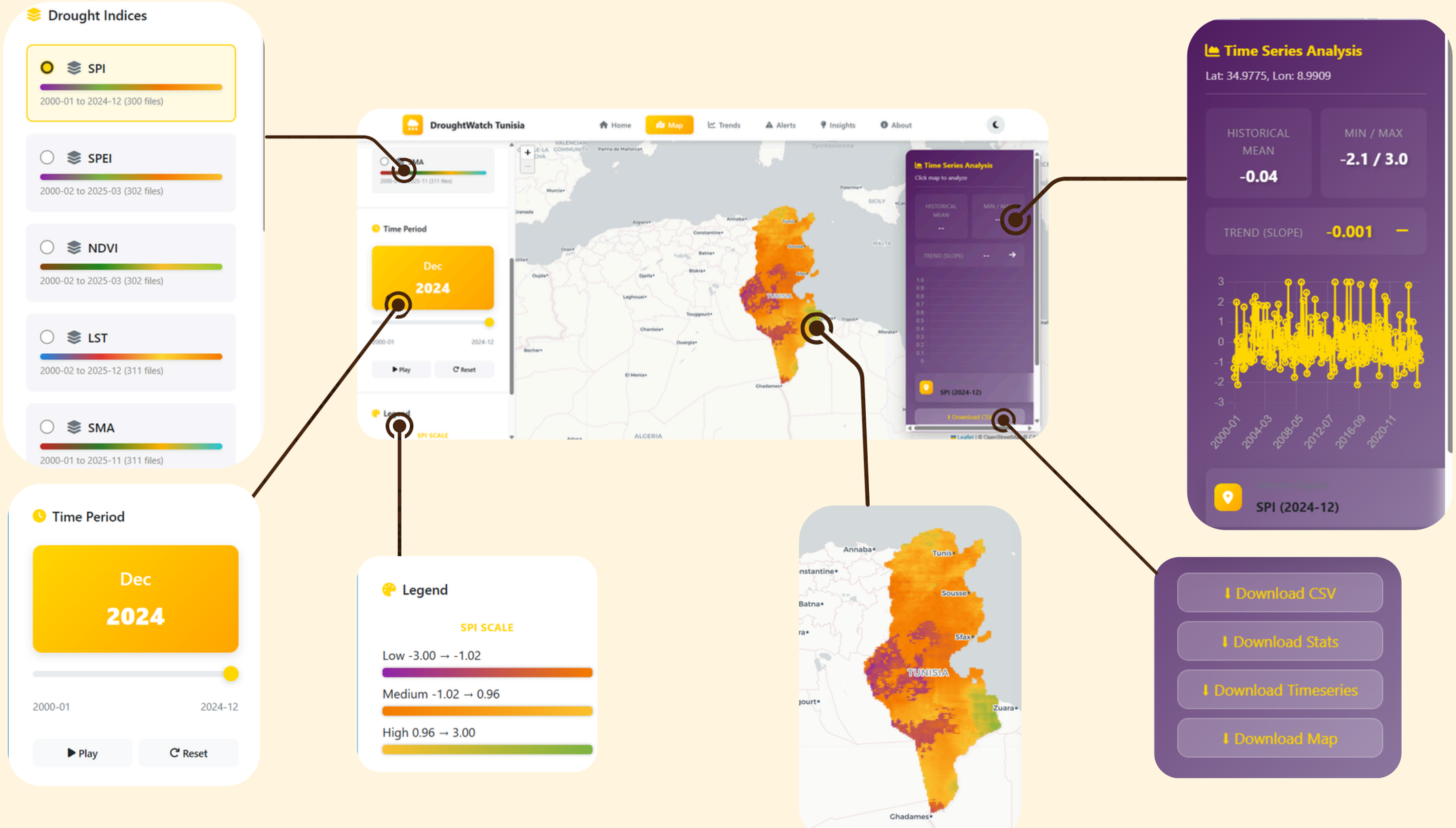

CSS


HTML


JS


Interactive map
Stats Panel
Control Panel
Export tools

4.DASHBOARD OVERVIEW



5.ADDED VALUE

Practical & Societal

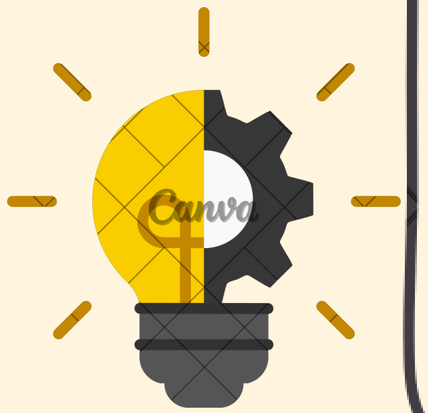
- Data accessibility
- Open environmental data

Scientific & Technical

- Multi-source drought indicators
- Automatic layer discovery
- Spatial and temporal analysis

Innovation

- Fully automated backend
- Dual extraction engine
- User-friendly API architecture



6.LIMITATIONS

Data

- Limited satellite data availability.
- Different spatial resolutions across layers.
- Requires external preprocessing (e.g., masking).
- No real-time data updates.

Technical

- Basic error handling.
- No user authentication.

Analytical

- Focus on visualization, not prediction.
- Lacks ground validation data.
- Uses simple linear trend estimation.



6.FUTURE WORK

Data Expansion

- Real-time data.
- Ground-based observations.
- New layers: poverty, land use/land cover, population, water demand...
- Harmonize datasets with different spatial resolutions.

Technical Enhancements

- Add a database for faster queries and scalable storage.
- Optimize backend for large raster processing.
- Implement user authentication and role-based access.

Analytical improvements

- AI/ML models
- Multi-factor risk assessment (socioeconomic/environmental layers).
- Automated drought/vulnerability alerts.



7.CONCLUSION

- DroughtWatch provides real-time drought monitoring in Tunisia.
- Integrates multiple drought indicators (SPI, SPEI, SMA, LST, NDVI).
- Enables interactive map visualizations and time-series analysis.
- Offers downloadable data and statistical summaries.
- Serves as a decision-support tool for water resource management and planning.



Thanks for your interest